

Aplicação de Machine Learning para detecção de ameaças em dispositivos de Internet das Coisas

Alessandra Carolina Domiciano e Ana Paula Serafim de Góis

Resumo—A análise de tráfego de rede baseada em Machine Learning (ML) fornece maior flexibilidade na identificação de ataques cibernéticos e suas variações, além de permitir otimizações para operação com baixa sobrecarga computacional. Este trabalho tem como objetivo construir um ambiente de testes realista para a avaliação do modelo proposto em [3], retreinado com o dataset MQTT-IoT-IDS2020. Para a avaliação prática, foi implementada uma rede IoT com dispositivos físicos e desenvolvido um sistema integrado capaz de realizar captura de tráfego, processamento dos dados e predição de classes em tempo real, utilizando uma abordagem binária (tráfego benigno vs. maligno). Durante os testes, foi gerado tráfego MQTT legítimo, além da execução de três tipos distintos de ataques. Dentre os modelos testados, o `xgb_full` destacou-se por apresentar o melhor equilíbrio entre as métricas e o maior F1-score, métrica priorizada neste trabalho. No entanto, a análise detalhada revelou um desempenho assimétrico entre as classes, com forte viés em favor da classe benigna e desempenho significativamente inferior na detecção da classe maligna — justamente a mais crítica em cenários de segurança. Por outro lado, os testes de latência demonstraram que o modelo é computacionalmente leve e viável para execução local em dispositivos embarcados como o Raspberry Pi 4, com tempo médio de predição de 1,4 μ s por amostra. Apesar disso, os resultados indicam que o sistema de detecção ainda demanda melhorias substanciais, especialmente em sua capacidade de identificar com precisão ataques reais.

Index Terms—IoT Security, Machine Learning, XGBoost, IDS

Abstract—Network traffic analysis based on Machine Learning (ML) provides greater flexibility in identifying cyberattacks and their variations, while also allowing optimizations for operation with low computational overhead. This work aimed to build a realistic test environment to evaluate the model proposed in [3], retrained using the MQTT-IoT-IDS2020 dataset. For practical evaluation, an IoT network with physical devices was implemented, and an integrated system was developed to perform traffic capture, data processing, and real-time binary classification, distinguishing between benign and malicious traffic. During testing, legitimate MQTT traffic was generated alongside three distinct types of attacks. Among the models tested, `xgb_full` stood out by presenting the best balance across the evaluated metrics and the highest F1-score, which is the prioritized metric in this study. However, a detailed analysis revealed a performance asymmetry between the classes, with a strong bias toward the benign class and significantly lower performance in detecting the malicious class — the most critical in security scenarios. On the other hand, latency tests showed that the model is computationally lightweight and feasible for deployment on edge devices such as the Raspberry Pi 4, with an average prediction time of 1.4 μ s per sample. Nevertheless, results indicate that the detection system still requires significant improvements, particularly regarding its ability to accurately identify real-world attacks.

Index Terms—IoT Security, Machine Learning, XGBoost, IDS

I. INTRODUÇÃO

O rápido crescimento do número de dispositivos IoT criou desafios para a detecção escalável de ameaças. Um dos mecanismos de segurança mais importantes para a defesa de uma rede ou de um host é o Sistema de Detecção de Intrusão (IDS). O IDS é capaz de monitorar e analisar o tráfego de rede em tempo real, detectando ameaças e atividades suspeitas. Tradicionalmente, ele se baseia em assinaturas de malwares e regras pré-definidas que descrevem padrões específicos de comportamento malicioso já conhecidos. Por isso, ele pode não ser tão eficaz contra ataques mais sofisticados e variações. Além disso, um IDS tradicional requer um elevado poder computacional para processar grandes volumes de dados e realizar comparações de assinaturas, o que o torna inadequado para dispositivos de IoT, que normalmente possuem recursos computacionais restritos.

Nesse sentido, a análise de tráfego de rede baseada em Machine Learning (ML) surge como uma alternativa, a fim de melhorar consideravelmente o desempenho da detecção de ameaças. Os modelos de ML têm a capacidade de aprender e se adaptar ao longo do tempo, o que resulta em uma detecção mais precisa de ataques e atividades suspeitas sem a necessidade de atualizações manuais constantes, permitindo que ele identifique uma ameaça mesmo que nunca tenha se deparado com ela anteriormente. Porém, existem desafios a serem superados no uso de ML para detecção de ameaças em dispositivos de IoT. Por exemplo, dados massivos e alta dimensionalidade fazem a análise em tempo real ter um custo computacional grande, enquanto o desbalanceamento de classes prejudica o desempenho dos modelos, tornando ameaças raras mais difíceis de detectar. Para execução em um dispositivo IoT, os modelos de ML devem ser leves e rápidos, com boa capacidade de generalização e desempenho equilibrado, com um bom compromisso entre sensibilidade e especificidade.

Várias pesquisas têm sido desenvolvidas para se obter modelos de ML que entreguem bom desempenho na detecção de ameaças de IoT, mas que cumpram com os requisitos impostos por tal cenário. Em [1], foi proposto um pipeline que integra técnicas de balanceamento de dados generativas e tradicionais com Otimização de Hiperparâmetros (HPO), melhorando o desempenho do AutoML em até 6%. O trabalho utilizou o dataset IoT-23 e modelos de árvore de decisão como Random Forest (RF) e Extra Trees, que alcançaram 99,86% de precisão, superando Redes Neurais Profundas (DNN) e Máquina de Vetores de Suporte (SVM). Em [2], um modelo

híbrido que integra Redes Neurais Convolucionais e Memória de Longo e Curto Prazo (CNN-LSTM) atingiu 99,995% de precisão para classificação binária e 99,96% para classificação multiclasse, utilizando o dataset CICIOT2023.

Em [3], que consiste no artigo base para este trabalho, é proposto um pipeline para otimização de hiperparâmetros para melhorar a acurácia de detecção e reduzir a sobrecarga computacional durante o treinamento. Assim, ele permite o treinamento em grandes conjuntos de dados, mantendo a eficiência sem exigir aceleração de GPU, o que o torna adequado para aplicações de IoT. O teste foi executado utilizando os algoritmos LightGBM, XGBoost e XGBoostRF com o dataset CICIOT23. Os experimentos avaliaram o desempenho com taxas de amostragem dos dados de 1%, 5% e 10%, além do dataset “cru” (sem implementar nenhuma otimização nos dados), para casos de 2, 8 e 34 classes. Com base nos resultados, os autores sugerem o XGBoost com amostragem de 10% como base para detecção de ameaças de IoT, embora o desempenho possa variar de acordo com o cenário de implantação.

Vários datasets de ameaças de IoT têm sido desenvolvidos e empregados em diversos trabalhos de pesquisa. O dataset MQTT-IoT-IDS2020 [4], utilizado neste trabalho, possui cinco tipos de cenário. Um deles consiste em tráfego normal e os outros quatro são ataques capturados de forma independente. Os ataques são varredura agressiva (Scan A), varredura de portas UDP (Scan sU), força bruta Sparta SSH (Sparta) e força bruta MQTT (MQTT BF).

Este trabalho tem como objetivo construir um ambiente de teste para avaliação da proposta [3]. Para tanto, foi implementada uma rede IoT com dispositivos físicos, que produzem 4 tipos de tráfego: MQTT (benigno) e três tipos distintos de ataque (maligno). Também foi desenvolvido um software em Python para captura do tráfego, processamento dos dados e predição de classes. Utilizou-se a abordagem de classificação binária, distinguindo entre tráfego benigno e maligno.

Este documento está estruturado da seguinte forma: na Seção I, o problema é introduzido e contextualizado, com a apresentação de abordagens relevantes da literatura, incluindo o trabalho-base desta pesquisa. A Seção II descreve a metodologia adotada. Na Seção III, são apresentados os resultados dos experimentos realizados, acompanhados de suas respectivas análises e discussões. Por fim, a Seção IV apresenta as conclusões, sintetizando os principais achados e indicando direções para trabalhos futuros.

II. METODOLOGIA

O desenvolvimento deste trabalho pode ser dividido em três etapas: provisionamento da rede IoT, desenvolvimento do sistema de teste, e treinamento do modelo.

A. Provisionamento da rede IoT

A rede IoT trata-se de uma WLAN cuja topologia e implementação são exibidas, respectivamente, na “Fig. 1” e na “Fig. 2”

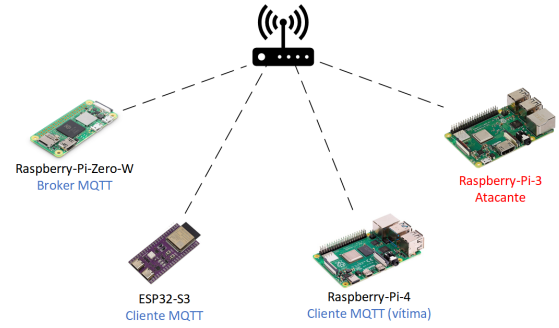


Figura 1. Topologia de rede IoT implementada para este trabalho.

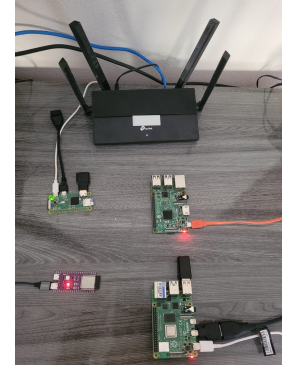


Figura 2. Rede IoT implementada para este trabalho.

A rede é composta por quatro dispositivos IoT, começando por uma Raspberry Pi Zero W, que executa um broker MQTT. Também há um ESP32, executando um cliente MQTT que publica dados em um tópico de teste e assina esse mesmo tópico. Além desses, existe uma Raspberry Pi 4, também executando um cliente MQTT que publica dados em um tópico de teste e assina esse mesmo tópico. Ela é a vítima dos ataques, por esse motivo, é quem executa o sistema de detecção de ameaças. Por fim, tem-se a Raspberry Pi 3, que é o atacante e, portanto, dispara os ataques contra a vítima. Os ataques Scan A e Scan sU foram realizados com a ferramenta Nmap, já o ataque Sparta foi feito com a ferramenta Hydra.

B. Desenvolvimento do sistema de teste

O fluxograma do sistema desenvolvido para teste do modelo é apresentado na “Fig. 3”.

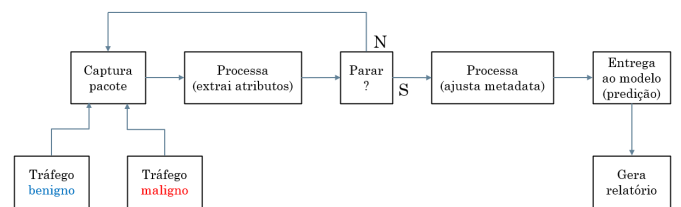


Figura 3. Sistema de detecção desenvolvido para teste do modelo.

O sistema inicia realizando a captura do tráfego de rede que entra e sai da interface do dispositivo vítima e prossegue

fazendo o processamento dos dados, que se trata da extração dos atributos. São extraídos 30 atributos, que consistem em campos de cabeçalho dos protocolos IP, TCP ou UDP e MQTT, conforme o padrão do dataset de treinamento. Esse processo é realizado pacote a pacote até que o usuário pare a captura. Uma vez que isso ocorre, o conjunto de dados é submetido a um novo processamento, desta vez, para ajustar os valores dos atributos e selecioná-los conforme o metadata utilizado no treinamento do modelo. Por fim, os dados são entregues ao modelo, que é integrado ao sistema, para realizar as predições e gerar as métricas de avaliação.

C. Treinamento do modelo

O modelo testado utiliza o XGBoost (Extreme Gradient Boosting), um algoritmo de ML bastante empregado para resolver problemas de classificação e regressão. Ele aplica árvores de decisão e é baseado na técnica de gradient boosting, que combina diversos modelos fracos para construir um modelo mais robusto e preciso. A implementação do modelo contempla etapas prévias de pré-processamento dos dados e um pipeline para otimização de hiperparâmetros (HPO), conforme proposto em [3]. Esses processos geraram 7 versões, que podem ser descritas da seguinte forma:

- 1) `xgb_full`: é a linha de base, utiliza o dataset puro.
- 2) `xgb_sampled`: realiza amostragem do dataset em 10% incluindo ao menos uma amostra de cada classe.
- 3) `xgb_reduced`: realiza amostragem do dataset e seleção de atributos preservando os que atingem mais de 95% de ganho de informação.
- 4) `xgb_filtered`: realiza amostragem do dataset, seleção de atributos e filtragem de amostras com base na média das frequências relativas dos valores dos atributos.
- 5) `xgb_best_latency`: realiza amostragem do dataset, seleção de atributos, filtragem de amostras e otimização de hiperparâmetros com objetivo de minimizar a latência de predição.
- 6) `xgb_best_f1_score`: realiza amostragem do dataset, seleção de atributos, filtragem de amostras e otimização de hiperparâmetros com objetivo de minimizar o F1 score.
- 7) `xgb_best_balance`: realiza amostragem do dataset, seleção de atributos, filtragem de amostras e otimização de hiperparâmetros com objetivo de equilibrar a latência de predição e o F1 score.

Para este trabalho, o modelo foi retreinado com o dataset MQTT-IoT-IDS2020 [4], com exceção dos dados do ataque MQTT BF. A abordagem de classificação foi a binária, com uma das classes sendo o tráfego normal (benigno) e a outra representando todos os ataques (maligno). Os dados do dataset são gerados a partir de uma rede simulada e são fornecidos com atributos processados no formato .csv. Empregou-se a versão com atributos baseados em pacote. O dataset é bastante desbalanceado e sua distribuição é ilustrada na “Fig. 4”.

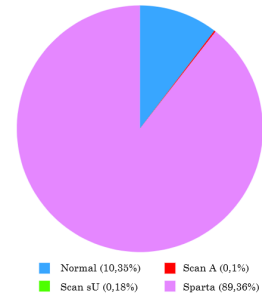


Figura 4. Distribuição dos dados de treinamento.

III. RESULTADOS E DISCUSSÃO

O experimento consistiu em enviar tráfego de rede benigno e maligno ao dispositivo vítima, a fim de verificar o desempenho do modelo em um cenário real e diferente daquele que gerou os dados de treinamento. Durante os testes, buscou-se produzir um equilíbrio entre as classes e entre os ataques. Para tanto, os ataques Sparta, Scan A e Scan sU foram executados uma, duas e duas vezes, respectivamente, enquanto o tráfego MQTT fluía continuamente. A distribuição final dos dados de teste pode ser observada na “Fig. 5”. Por fim, as matrizes de confusão e as métricas de desempenho, acurácia, precisão, recall e F1 score, além da latência de predição, foram calculadas para cada versão do modelo.

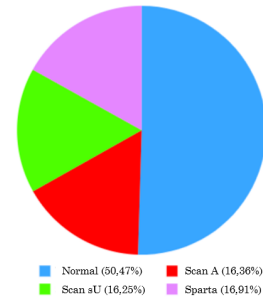


Figura 5. Distribuição dos dados de teste.

De forma geral, os modelos `xgb_best_latency`, `xgb_best_f1_score` e `xgb_best_balance` apresentaram desempenho insatisfatório, com acurácia e F1-score abaixo de 50%. Diante desses resultados, optou-se por concentrar a análise das métricas nos modelos `xgb_full`, `xgb_sampled`, `xgb_reduced` e `xgb_filtered`, com ênfase no primeiro, que demonstrou desempenho superior em relação aos demais segundo a métrica de F1 score. A “Fig. 6” apresenta uma síntese comparativa das métricas de desempenho de cada modelo.

De maneira geral, os quatro modelos avaliados apresentaram desempenhos semelhantes, com pequenas variações entre eles. Na ordem `xgb_full`, `xgb_sampled`, `xgb_reduced` e `xgb_filtered`, as precisões médias foram de 78,0%, 72,9%, 78,1% e 78,8%, respectivamente. Já o recall variou de 65,7% no `xgb_full` até 60,6% no `xgb_reduced`, com valores intermediários de 62,9%

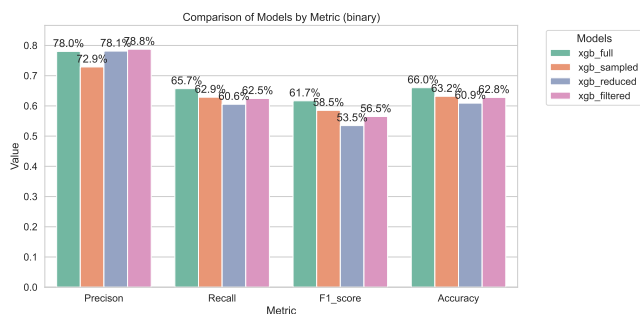


Figura 6. Métricas para classificação binária dos modelos com melhor desempenho.

e 62,5% para os modelos xgb_sampled e xgb_filtered. Em relação ao F1-score, que é a métrica priorizada neste trabalho, os resultados foram mais baixos: 61,7% (xgb_full), 58,5% (xgb_sampled), 53,5% (xgb_reduced) e 56,5% (xgb_filtered). A acurácia global dos modelos seguiu a mesma tendência, com valores de 66,0%, 63,2%, 60,9% e 62,8%, respectivamente.

Esses valores representam métricas médias e, portanto, não capturam completamente as diferenças de desempenho entre as classes. Para uma compreensão mais aprofundada do comportamento dos modelos, torna-se necessária a análise das métricas específicas por classe. Para isso, optou-se por utilizar o modelo xgb_full, uma vez que ele apresentou o melhor equilíbrio geral entre as métricas avaliadas, além de ter alcançado o maior F1-score, métrica priorizada neste trabalho por refletir o compromisso entre precisão e recall. A "Fig. 7" apresenta os resultados de predição do modelo xgb_full para ambas as classes (maligna e benigna).

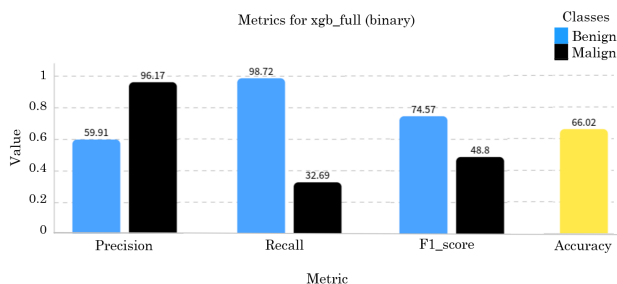


Figura 7. Métricas de classificação do modelo xgb_full

A análise detalhada do modelo xgb_full revela um desempenho significativamente desequilibrado entre as classes. Observa-se um forte viés em favor da classe benigna, evidenciado pelo seu elevado recall (98,7%), o que indica que quase todas as amostras benignas foram corretamente classificadas. No entanto, a precisão associada a essa classe é relativamente baixa (59,9%), sugerindo que o modelo produz um número considerável de falsos positivos, ou seja, casos malignos erroneamente classificados como benignos. Esse desequilíbrio entre precisão e recall resulta em um F1-score de 74,6% para a classe benigna, valor que, embora possa ser considerado

razoável, mascara a fragilidade do modelo em relação à outra classe.

Por outro lado, o desempenho para a classe maligna é bastante insatisfatório. Embora a precisão seja elevada (96,2%), o recall é extremamente baixo (32,7%), indicando que o modelo deixa de identificar mais de dois terços das amostras malignas. Isso implica uma alta taxa de falsos negativos. O F1-score para essa classe é de 48,8%, refletindo um desempenho baixo em relação a essa classe.

No que se refere à acurácia global do modelo, observa-se um valor relativamente baixo, em torno de 66,0%. É importante destacar que essa métrica não representa adequadamente a qualidade preditiva do modelo, pois, nesse caso, ela é inflada pelo bom desempenho sobre a classe benigna, enquanto ignora o baixo desempenho sobre a classe maligna. Assim, não se deve utilizá-la isoladamente.

O contraste entre o desempenho em ambas as classes torna-se mais evidente por meio da matriz de confusão, apresentada na "Fig. 8". Observa-se um baixo número de falsos negativos, o que à primeira vista sugere bom desempenho; no entanto, há um elevado número de falsos positivos, indicando que o modelo frequentemente classifica erroneamente casos malignos como benignos. Essa limitação compromete seriamente a confiabilidade do modelo em cenários críticos, como a detecção de ameaças.

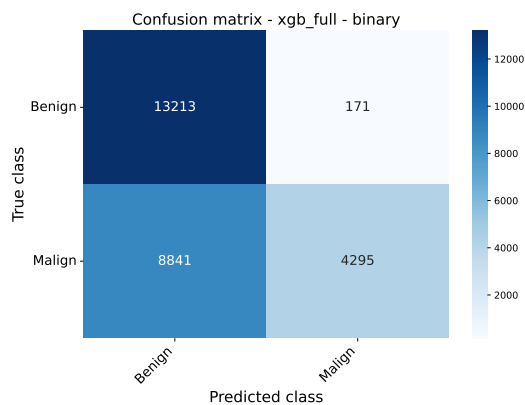


Figura 8. Matriz de confusão do modelo xgb_full

Além disso, os dados de predição gerados pelo modelo xgb_full foram armazenados em arquivo para permitir uma análise posterior do desempenho por tipo de ataque. Para isso, os dados foram segmentados em três subconjuntos, cada um contendo amostras de um único ataque específico, juntamente com tráfego benigno. A proporção entre classes em cada subconjunto foi mantida aproximadamente equilibrada, com cerca de 50% de amostras benignas. As métricas de desempenho do modelo xgb_full para cada um dos ataques são apresentadas na "Fig. 9".

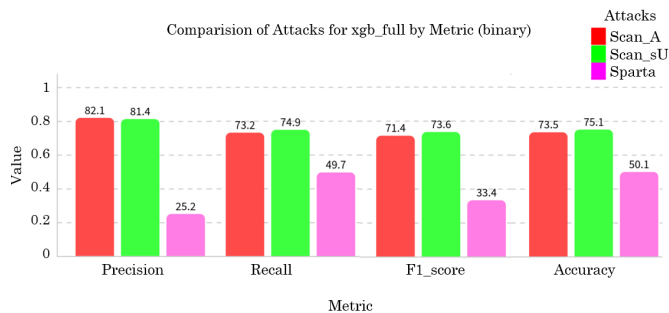


Figura 9. Métricas de desempenho por ataque.

Com base no gráfico, observa-se que o modelo apresenta um desempenho razoável nos testes com os ataques isolados Scan A e Scan U, com valores de precisão de 82,1% e 81,4%, recall de 73,2% e 74,9%, F1-score de 71,4% e 73,6% e acurácia de 73,5% e 75,1%, respectivamente. Entretanto, é importante salientar que tais métricas representam valores médios agregados, os quais podem ocultar desequilíbrios significativos no desempenho entre as classes benigna e maligna, conforme discutido na análise geral do modelo.

Com relação ao teste com o ataque isolado ataque Sparta, os resultados indicam um desempenho significativamente inferior. Todas as métricas ficaram abaixo de 50%, com exceção da acurácia em 50,1%, o que equivale a um classificador aleatório. Isso evidencia que o modelo tem uma dificuldade grande em identificar esse tipo de ataque.

Também foi avaliada a latência de predição de cada modelo, conforme apresentado na "Fig. 10". Todos os modelos tiveram bom desempenho nesse quesito. O xgb_full, foco da análise deste trabalho, apresentou latência média de 1,4us por amostra, o que é compatível com um cenário de detecção em tempo real. No entanto, esse valor corresponde apenas à predição da amostra já processada, sem levar em conta o processo de captura e processamento dos dados. Para analisar a viabilidade do sistema como um todo, é necessário incluir esses processos na análise.

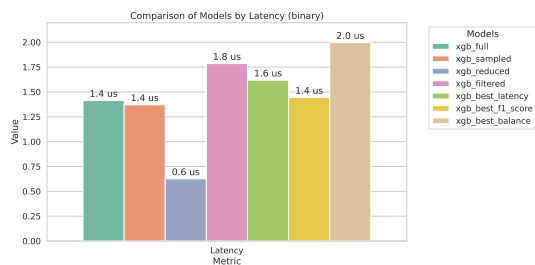


Figura 10. Latência média por amostra (us) dos modelos.

IV. CONCLUSÃO

Os experimentos realizados permitiram avaliar o desempenho de diferentes versões do modelo XGBoost em um cenário real de detecção de ataques em redes IoT, utilizando

um ambiente com tráfego benigno e três tipos distintos de ataques (Sparta, Scan A e Scan U). Entre os modelos testados, o xgb_full destacou-se por apresentar o melhor equilíbrio entre as métricas avaliadas, especialmente no que se refere ao F1-score, métrica priorizada neste trabalho por considerar simultaneamente precisão e recall.

Apesar disso, a análise detalhada revelou um desempenho assimétrico entre as classes, com o modelo favorecendo fortemente a classe benigna. Esse comportamento, embora contribua para uma acurácia global aparentemente razoável (66,0%), oculta um desempenho significativamente inferior na detecção da classe maligna, que é justamente a mais crítica em cenários de segurança. A baixa capacidade de identificar amostras de ataque, evidenciada pelo recall de apenas 32,7% e F1-score de 48,8% nessa classe e reforçada na análise por ataque, indica uma limitação importante do modelo na detecção de ameaças reais.

Por outro lado, os testes de latência demonstraram que o modelo é computacionalmente leve e viável para execução em dispositivos embarcados como o Raspberry Pi 4, com tempo médio de predição de 1,4 us por amostra, embora destaque-se que esse valor não inclui a etapa de captura e processamento de pacotes, que deve ser incorporada em futuras avaliações para uma análise completa da viabilidade do sistema em ambiente produtivo.

Em suma, foi criado um cenário realista e funcional para a execução de testes de detecção de ameaças cibernéticas em ambientes IoT, demonstrando a viabilidade de aplicação local de modelos de machine learning para tal finalidade. No entanto, os resultados evidenciam que o sistema de detecção ainda demanda melhorias significativas, sobretudo em relação à sua capacidade de identificar com precisão os ataques, devido à alta taxa de falsos positivos. Como trabalho futuro, propõe-se a integração da dos processos de captura e pré-processamento de pacotes ao fluxo de predição, com o intuito de incluir essas etapas ao cálculo de latência média por amostra, o uso de outros bancos de dados e ameaças de IoT e a utilização de outros tipos de modelos, como os de IA generativa.

REFERÊNCIAS

- [1] M. V. C. Aragão, M. F. Carvalho, T. M. Pereira et al. (2024, May) Enhancing AutoML Performance for Imbalanced Tabular Data Classification A Self-Balancing Pipeline. [Online]. Available: <https://www.researchsquare.com/article/rs-4469436/v1>.
- [2] S. Yaras and M. Dener, "IoT-Based Intrusion Detection System Using New Hybrid DL Algorithm," *Electronics*, vol. 13, no. 6, p. 1053, 2024.
- [3] M. V. C. Aragão, T. M. Pereira et al. (2025) A Sample-Based, Multi-Stage Machine Learning Pipeline for Scalable IoT Threat Detection, doi: <https://doi.org/10.1109/LES.2025.3567025>
- [4] Hanan Hindy, Christos Tachtatzis, Robert Atkinson, Ethan Bayne, Xavier Bellekens, June 23, 2020, "MQTT-IoT-IDS2020: MQTT Internet of Things Intrusion Detection Dataset", IEEE Dataport, doi: <https://dx.doi.org/10.21227/bhxy-ep04>.
- [5] M. V. C. Aragão, S. Mafra, e F. A. P. de Figueiredo, "Análise de Tráfego de Rede com Machine Learning para Identificação de Ameaças a Dispositivos IoT" *Simpósio Brasileiro de Telecomunicações e Processamento de Sinais (SBrT)*, 2022. <https://dx.doi.org/10.14209/sbrt.2022.1570824939>